Lembar Kerja Praktikum P07 – Exception Handling

Lembar kerja ini menggantikan laporan praktikum. Isi **sambil** mengerjakan setiap langkah di modul, lalu kumpulkan di akhir sesi sebagai `LK-P07-<NIM>.pdf` bersama zip kode Anda.

Petunjuk pengisian

- Jawab singkat dan jelas. Satu sampai tiga kalimat per pertanyaan sudah cukup.
- Tempel output program sebagai **teks**, bukan tangkapan layar.
- Di L2 dan L3, isi kolom **prediksi** sebelum menjalankan program. Prediksi yang salah tidak mengurangi nilai; yang dinilai adalah penjelasan selisihnya.
- Total nilai LK 50 poin. Bagian L0 wajib diisi, tanpa poin.

L0 – Identitas dan parameter (wajib)

Isian	Jawaban
Nama	Rifqi Aditya
NIM	2511533002
Kelas	Informatika B
Tanggal praktikum	9 – 11 – 2026
Versi Java (baris pertama `java -version`)	java version "25.0.5.0.3" 2026-10-25 LTS
Saldo awal e-wallet (`NimParams`)	Rp 250000
Stok awal KOPI-001	4
Stok awal RDG-002	1

L1 – Membaca crash dan menangani input (8 poin)

(a) Bedah stack trace `CrashDemo` skenario 2 dan 3 (4 poin). Contoh pengisian untuk skenario 1 ada di modul.

Yang dicari	Skenario 2	Skenario 3
Jenis exception	Exception in thread "main" java.lang.NumberFormatException: For input string: "dua"	Exception in thread "main" java.lang.NullPointerException: Cannot invoke "shop.model.Address.getCity()" because the return value of "shop.demo.CrashDemo$Customer.getAddress()" is null
Checked atau unchecked?	Unchecked	Unchecked
Pesan	For input string: "dua" : untuk string input : “dua”	Cannot invoke "shop.model.Address.getCity()" because the return value of "shop.demo.CrashDemo$Customer.getAddress()" is null : tidak bisa memanggil "shop.model.Address.getCity()" karena nilai yang dikembalikan dari "shop.demo.CrashDemo$Customer.getAddress()" Adalah null

Yang dicari	Skenario 2	Skenario 3
Baris kode Anda yang pertama (file:baris, method)	Baris 28 : return Integer.parseInt(text);	Baris 32 : return customer.getAddress().getCity();
Dipanggil dari (file:baris, method)	Baris 43 : case "2" -> System.out.println("Jumlah: " + readQuantity("dua"));	Baris 44 : case "3" -> System.out.println("Kirim ke: " + shippingCity(new Customer("Siti")));
Apa penyebabnya, dengan kata-kata Anda sendiri?	readQuantity "dua" meneruskan teks yang bukan angka ke Integer.parseInt, dan parseInt langsung melempar NumberFormatException. Karena readQuantity maupun main tidak punya try catch, exception itu naik sampai ke JVM, program mati di tengah jalan, dan baris "Selesai" tidak pernah dicetak	Objek Customer("Siti") dibuat tanpa Address, jadi field address nya tetap null. Begitu getAddress() mengembalikan null, pemanggilan .getCity() di baris itu menabrak null dan Java melempar NullPointerException.

(b) Jumlah `-2` (2 poin). Sebelum `ShopApp` diperbaiki, apa yang terjadi saat Anda memilih produk 2 lalu mengetik jumlah `-2`? Mengapa tidak muncul `NumberFormatException`? Apa beda "format input salah" dengan "aturan bisnis dilanggar"?

Programnya jalan terus tanpa exception apa pun. Layar tetap mencetak " + -2 x Rendang Kemasan 250g", tetapi keranjang ternyata kosong dan baris terakhirnya "Pembayaran berhasil? false". "-2" adalah teks bilangan bulat, jadi Integer.parseInt mengembalikan -2 tanpa protes dan NumberFormatException tidak pernah muncul. Perulangan for “i < quantity” berhenti sebelum putaran pertama karena 0 < -2 bernilai false; keranjang tidak bertambah. Format input salah berarti teksnya tidak bisa dibaca sebagai angka, dan parseInt sendiri yang melapor lewat NumberFormatException.

(c) `if` atau `catch`? (2 poin). Mengapa nomor produk di luar katalog dicegah dengan `if`, sedangkan teks yang bukan angka ditangkap dengan `catch (NumberFormatException _)`?

Nomor di luar katalog bisa dicek dan pasti sebelum dipakai “choice < 1 || choice > catalog.size()”, jadi cukup diveri dengan if. Exception karena lebih cocok untuk keadaan tak terduga,

L2 – Prediksi dulu, baru jalankan (12 poin)

Isi kolom **Prediksi** sebelum menjalankan program. Tulis baris-baris output sesuai urutan, misalnya `A1, A2, A4, A5`.

`FlowDemo` bagian A, B, C (7 poin: A 2, B 3, C 2)

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
A: `partA("12")` lalu `partA("12rb")`	partA("12"): A1, A2, A4, A5 partA("12rb"): A1, A3, A4, A5	A1 mulai A2 jumlah = 12 A4 finally A5 lanjut A1 mulai A3 bukan angka A4 finally A5 lanjut	ya	Untuk "12", parseInt lolos dan blok catch dilewati, tetapi finally tetap dijalankan A4 sebelum A5. Untuk "12rb", parseInt gagal dengan NumberFormatException di baris kedua try, sehingga A2 tidak tercetak; catch mengambil alih dan mencetak A3, finally mencetak A4, lalu A5 muncul karena exception-nya sudah beres ditangani.
B: `partB(300000, "3")` lalu `partB(300000, "0")`	partB(300000,"3"): B1, B2, B4, lalu B5 hasil = 100000. partB(300000,"0"): B1, B4, lalu B7 main menangkap: / by zero. B3, B5 kedua, dan B6 tidak tercetak.	B1 bulan = 3 B2 cicilan = 100000 B4 finally B5 hasil = 100000 B1 bulan = 0 B4 finally B7 main menangkap: / by zero	ya	Pada "3", nilai return sudah selesai dihitung lebih dulu, lalu finally mencetak B4 sebelum partB benar-benar keluar. println di main baru berjalan sesudah partB kembali, makanya B4 tampil mendahului B5. Pada "0", total / months dengan months = 0 melempar ArithmeticException; catch NumberFormatException tidak cocok sehingga B3 tidak muncul, namun finally tetap jalan dan mencetak B4. Exception lalu naik ke main dan ditangkap di sana (B7), jadi B5 yang kedua dan B6 tidak sempat tercetak.
C: `partC(5)`	C1, C2, C3 sisa = 4	C1 cek stok C2 finally C3 sisa = 4	ya	Nilai return “stock - 1 = 4” sudah dihitung sebelum finally jalan; finally mencetak C2 lebih dulu, baru method kembali ke pemanggil dan C3 dicetak. Jadi finally tetap dieksekusi walaupun ada return di atasnya.

`CariProdukDemo` bagian D (3 poin)

Perintah	Prediksi output	Output sebenarnya	Penjelasan: catch mana yang jalan, dan mengapa
`...` `CariProdukDemo 2`	D1 SKU: RDG-002 D5 pencarian selesai	D1 SKU: RDG-002 D5 pencarian selesai	Tidak ada exception di sini, karena skus [2 - 1] adalah RDG-002 dan itu langsung tercetak sebagai D1. Semua catch dilewati, sedangkan finally tetap jalan seperti biasa (D5)
`...` `CariProdukDemo abc`	D2 nomor harus bilangan bulat D5 pencarian selesai	D2 nomor harus bilangan bulat D5 pencarian selesai	parseInt "abc" melempar NumberFormatException, dan catch yang jalan adalah D2, bukan D4. Program membaca catch dari atas ke bawah dan hanya menjalankan yang pertama cocok; D2 ada di atas D4, jadi meskipun NumberFormatException sebenarnya juga turunan IllegalArgumentException
`...` `CariProdukDemo 8`	D3 nomor di luar katalog: Index 7 out of bounds for length 3 D5 pencarian selesai	D3 nomor di luar katalog: Index 7 out of bounds for length 3 D5 pencarian selesai	Skus [8 - 1] sama dengan skus[7], sementara array-nya cuma berisi 3 elemen, jadi terlempar ArrayIndexOutOfBoundsException. Yang menangkap adalah catch D3, dan teks "Index 7 out of bounds for length 3" datang dari e.getMessage()
`...` `CariProdukDemo` (tanpa argumen)	D4 Berikan satu nomor produk. D5 pencarian selesai	D4 Berikan satu nomor produk. D5 pencarian selesai	Tanpa argumen, args.length == 0 bernilai true, sehingga kode sendiri yang melempar IllegalArgumentException. Exception itu bukan NumberFormatException dan bukan ArrayIndexOutOfBoundsException, jadi turun ke catch D4.

L2.c – Ubah urutan catch (2 poin). Tempel pesan error compiler setelah catch `IllegalArgumentException` dipindah ke paling atas, lalu jelaskan artinya dalam satu kalimat.

Pesan Error : Unreachable catch block for NumberFormatException. It is already handled by the catch block for IllegalArgumentException

Artinya : Catch NumberFormatException tidak mungkin pernah dijalankan: IllegalArgumentException di atasnya sudah menangkap semua turunannya, termasuk NumberFormatException. Makanya catch yang lebih spesifik wajib ditulis lebih dulu

L3 – Custom exception dan checkout (10 poin)

(a) Error berantai (2 poin). Setelah `PaymentMethod.pay` diubah menjadi `void ... throws PaymentFailedException`, berapa error yang muncul, dan di file mana saja? Mengapa satu perubahan di interface memengaruhi file-file itu?

Jumlah error: File:
10 error, di file : EWalletPayment.java (3 error), BankTransferPayment.java (3 error), CreditCardPayment.java (3 error), dan CheckoutService.java (1 error).

Alasan:
PaymentMethod itu termasuk interface, jadi begitu signature pay-nya berubah, siapa pun yang terhubung ikut kena. EWalletPayment, BankTransferPayment, dan CreditCardPayment masih menulis pay yang mengembalikan boolean, sehingga tidak lagi sesuai dengan void throws PaymentFailedException. CheckoutService juga masih membaca hasil pay sebagai boolean. Intinya, mengubah hubungan memaksa semua class yang mengimplementasikan maupun memakainya diubah juga.

(b) Enam skenario `ScenarioRunner` (4 poin). Isi kolom prediksi dulu (pakai parameter L0), baru jalankan. Hasil diisi salah satu dari `[OK]`, `[SALDO]`, `[BAYAR]`, `[STOK]`, `[INVALID]`.

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S1 Kopi + Rendang, Reguler, e-wallet	[OK]	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	[OK] E-Wallet DompetKu membayar Rp215000	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	Ya
S2 Kopi x3, Reguler, transfer bank	[BAYAR]	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	[BAYAR] Bank Nagari sedang gangguan, coba lagi nanti	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	Ya
S3 Rendang x2, Reguler, e-wallet	[STOK]	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	[STOK] Stok RDG-002 tidak cukup: diminta 1, tersedia 0	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	Ya
S4 Kopi, Ekspres, kartu kredit	[SALDO]	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	[SALDO] Saldo tidak cukup: perlu Rp103000, tersedia Rp100000 (kurang Rp3000)	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	Ya
S5 Keranjang kosong	[INVALID]	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	[INVALID] IllegalStateException: Cart is empty	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	ya
S6 Teh x2, Reguler, e-wallet	[SALDO]	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	[SALDO] Saldo tidak cukup: perlu Rp80000, tersedia Rp35000 (kurang Rp45000)	KOPI 3 / RDG 0 / TEH 5 / saldo Rp35000	Ya

Pilih satu skenario yang prediksinya meleset (atau, jika semua cocok, skenario yang paling menarik), lalu jelaskan mengapa hasilnya seperti itu. Sebut juga mengapa pesan `[STOK]` menyebut "diminta 1".

Semua prediksi saya cocok, jadi saya ambil S3 (Rendang x2) karena paling menarik dibahas. Stok awal RDG-002 untuk NIM saya cuma 1, dan S1 sudah memakainya, jadi saat S3 jalan stok tinggal 0. checkout memesan barang satu per satu lewat inventory.reserve(sku, 1); rendang pertama saja sudah gagal, sehingga pesannya "diminta 1, tersedia 0" dan bukan "diminta 2". Stok dan saldo tidak berubah.
Kasus S2 (Kopi x3, transfer bank sedang gangguan) berbeda: ketiga kopi sempat dipesan, lalu dikembalikan begitu pembayaran gagal, sehingga KOPI tetap 3

(c) Mengembalikan stok lalu `throw e` (2 poin). Di method `checkout`, mengapa blok catch mengembalikan stok lalu melempar ulang exception yang sama, bukan mencetak pesan untuk pembeli? Apa yang terjadi pada stok jika `release` dihapus?

checkout hanya bertugas mengurus stok dan pembayaran, misal program cuma mencetak pesan atau mengembalikan nilai, ShopApp.buy tidak akan tahu bahwa checkout gagal, apalagi memilih antara [SALDO], [BAYAR], dan [STOK]. Jadi stok dikembalikan lebih dulu, lalu exception yang sama dilempar ulang dengan throw e, dan pemanggillah yang menentukan apa yang dibaca pembeli

(d) Urutan catch di `buy` (2 poin). Mengapa catch `InsufficientBalanceException` harus ditulis di atas catch `PaymentFailedException`? Apa yang terjadi jika urutannya ditukar?

InsufficientBalanceException turunan dari PaymentFailedException. Karena catch diperiksa dari atas dan hanya yang pertama cocok yang dijalankan, versi yang lebih spesifik harus ditaruh di atas.

L4 – Katalog dari file (6 poin)

(a) Baris katalog yang dilewati (3 poin).

Baris ke-	Isi baris	Apa masalahnya?	Exception yang tertangkap (nama class)
	KUE-003;Kue Lapis Legit;95rb;800	Harga "95rb" bukan bilangan bulat	NumberFormatException (ditangkap oleh catch IllegalArgumentException, karena turunannya)
	BATIK-005;Kain Batik Tulis	Hanya 2 kolom, seharusnya 4 “sku;nama;harga;beratGram”	IllegalArgumentException (expected 4 fields, found 2), dilempar oleh method parse
	SAMB-006;Sambal Lado Mudo 200g;-45000;250	Harga negatif	IllegalArgumentException (Price must not be negative), dilempar oleh konstruktor Product

(b) Ketika file tidak ada (3 poin). Tempel output `java -cp bin shop.app.ShopApp katalog-cadangan.csv`. Apa isi `e.getMessage()` dan `e.getCause()`? Kalau `ShopApp` hanya mencetak `e.getCause().getMessage()`, apa yang dilihat pengguna, dan mengapa itu kurang membantu?

```
PS D:\Ditzz\VisualStudio\Praktikum-07-Exception\Exception-2511533002> java -cp bin shop.app.ShopApp katalog-cadangan.csv
Toko tidak bisa dibuka: Katalog gagal dibaca: katalog-cadangan.csv
Penyebab asli: java.nio.file.NoSuchFileException: katalog-cadangan.csv
```

Jawaban: e.getMessage() berisi "Katalog gagal dibaca: katalog-cadangan.csv", pesan yang kita tulis sendiri di CatalogReader. Sedangkan e.getCause() mengembalikan java.nio.file.NoSuchFileException: katalog-cadangan.csv, yaitu IOException asli yang dibungkus oleh exception.
Kalau ShopApp hanya mencetak e.getCause().getMessage(), pengguna cuma melihat "katalog-cadangan.csv", sekadar nama file. Ia tidak tahu apa yang gagal (membaca katalog) ataupun sebabnya, sehingga pesannya nyaris tak berguna

L5 – Code review FragileShop (10 poin)

Tabel temuan (8 poin, 2 per method).

Method	Apa masalahnya?	Dampaknya (bagi pembeli atau programmer)	Perbaikan
`payOrder`	Blok catch kosong menelan PaymentFailedException, jadi kegagalan bayar lenyap tanpa meninggalkan jejak apa pun	Pembeli dan program sama-sama mengira pembayaran sukses "Bayar Rp85000 selesai", padahal saldo Rp50000 tidak berkurang. Pesanan bisa terproses tanpa pernah terbayar, dan programmer akan kesulitan menelusuri sumber bug nya	Buang try-catch dan tambahkan throws PaymentFailedException pada method nya. Di pemanggil main, tangkap exception itu lalu cetak "Pembayaran gagal: <pesan>".
`isInStock`	Exception dipakai sebagai jawaban ya/tidak, dan reserve + release punya efek samping pada stok padahal tujuannya cuma mengecek	Stok ikut berubah hanya karena ada pengecekan. Kalau ada yang bermasalah di antara reserve dan release, stok bisa jadi salah. Kodenya juga lebih lambat dan lebih membingungkan dibaca	Tanyakan langsung tanpa efek samping : return inventory.available(sku) > 0;
`readCatalogHeader`	Penyebab aslinya terbuang karena CatalogLoadException dibuat dengan cause null	Programmer hanya melihat "Katalog gagal dibaca" dengan penyebab: null, sehingga tidak bisa membedakan file yang tidak ada, izin baca yang kurang, atau file yang rusak	Teruskan exception aslinya: new CatalogLoadException("Katalog gagal dibaca: " + file, e)

Method	Apa masalahnya?	Dampaknya (bagi pembeli atau programmer)	Perbaikan
`addToCart`	Salah ketik pembeli (NumberFormatException) dan stok habis ditangani dengan cara dan pesan yang sama, sehingga pesan teknis ikut sampai ke pembeli.	Pembeli melihat "Gagal: For input string: "dua"", pesan teknis yang tidak ia mengerti dan tidak memberi tahu apa yang harus dilakukan. Multi-catch juga mengaburkan perbedaan jenis masalahnya	Pisahkan penanganannya: catch NumberFormatException "Jumlah harus berupa angka.", cek qty <= 0 dengan if, dan catch OutOfStockException dengan pesan yang menyebut stoknya

Output `FragileShop` setelah `payOrder` dan `readCatalogHeader` diperbaiki (2 poin).

Output:

```
D:\Ditzz\VisualStudio\Praktikum-07-Exception\Exception-2511533002> cmd /C "C:\Users\rifqi\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64
\bin\java.exe -XX:+ShowCodeDetailsInExceptionMessages -cp D:\Ditzz\VisualStudio\Praktikum-07-Exception\Exception-2511533002\bin review.FragileShop "
1) Pembayaran gagal: Saldo tidak cukup: perlu Rp85000, tersedia Rp50000
2) Kopi tersedia? true
3) Gagal: For input string: "dua"
   Gagal: Stok KOPI-001 tidak cukup: diminta 5, tersedia 1
4) Katalog gagal dibaca: katalog-lama.csv | penyebab: java.nio.file.NoSuchFileException: katalog-lama.csv
```

Penutup

Hasil SelfCheck terakhir. Salin dua baris berikut dari output SelfCheck.

=== HASIL: ...

```
=== HASIL: 24 / 24 pemeriksaan lulus (nilai SelfCheck 50 / 50) ===
```

=== TANTANGAN: ...

Refleksi (4 poin).

1. (2 poin) Mengapa kesalahan ketik pembeli (misalnya `dua` sebagai jumlah) perlu ditangani berbeda dari stok yang habis? Hubungkan dengan pilihan checked/unchecked.

Jawaban: Salah ketik adalah kesalahan input yang bisa dicegah dan diperbaiki langsung di tempat input, jadi exception unchecked bawaan Java (NumberFormatException) sudah cukup; pembeli tinggal diminta mengetik ulang. Stok habis lain urusannya. Itu kegagalan bisnis yang berada di luar program, tetapi pembeli masih bisa menganatsinya dengan mengurangi jumlah atau memilih produk lain. Karena itu OutOfStockException (extends ShopException) dibuat checked, supaya compiler memaksa pemanggil menanganinya dan stok yang sempat dipesan dikembalikan

2. (1 poin) Apakah pesan exception teknis, misalnya `For input string: "dua"`, sebaiknya ditampilkan langsung kepada pembeli? Beri satu contoh dari praktikum ini.

Jawaban: Tidak. Pesan tersebut membingungkan pembeli sekaligus membuka detail internal program. Contohnya addToCart di FragileShop, yang menampilkan "Gagal: For input string: "dua"". Pembeli sebenarnya cukup melihat "Jumlah harus berupa angka." seperti di ShopApp, sementara pesan teknisnya disimpan untuk programmer lewat log atau getCause

3. (1 poin, dinilai penuh asal diisi jujur) Satu hal tentang exception yang masih membingungkan atau ingin Anda tanyakan di pertemuan berikutnya.

Jawaban: